

Work Division and Integration Plan

Who builds what, how to build it, and how the ten pieces come together

Document	02 · Work Division and Integration Plan (v1.0)
Read first	Document 01 · Project Documentation (data model §8, API §9, AI design §10)
Team	10 members — 1 architecture/integration, 1 security, 3 backend modules, 3 AI, 2 frontend
Date	06 October 2026

Contents

1. Team at a glance
2. Ground rules every member follows
3. Member 1 — Architecture, contracts and integration
4. Member 2 — Authentication, sessions, roles and audit
5. Member 3 — Catalog and inventory API
6. Member 4 — Reservations and reading lists API
7. Member 5 — Clubs, meetings and reading progress API
8. Member 6 — AI foundations and semantic discovery
9. Member 7 — Spoiler-aware RAG (book chat)
10. Member 8 — Reading agent workflow
11. Member 9 — Frontend A: reader experience
12. Member 10 — Frontend B: clubs, staff and agent screens
13. Integration plan
14. Git workflow — step by step
15. Dependency map
16. Definition of done and review checklist
17. Communication and tracking

How to use this document: 1) Read Sections 1-2 (everyone). 2) Go to **your own member section** — it has your job, your folders, and numbered steps; each step is one Git branch. 3) Do the steps in order, ticking the *Done when* boxes. 4) Use Section 14 whenever you're unsure about Git.

1. Team at a glance

#	Role	Owns	Name
1	Architecture & Integration	shared/, middleware/error+validate, seed, CI, E2E, demo	_____
2	Auth, Sessions, Roles & Audit	modules/auth, modules/users, middleware/auth+roles+csrf+rateLimit, modules/audit	_____
3	Catalog & Inventory API	modules/books, modules/inventory, modules/catalogSources	_____
4	Reservations & Reading Lists API	modules/reservations, modules/readingLists, jobs/expireHolds	_____
5	Clubs, Meetings & Progress API	modules/clubs, memberships, meetings, progress, excerpt approval	_____
6	AI Foundations & Semantic Discovery	ai/ollama, ai/embeddings, vector indexes, ingestion, ai/discovery	_____
7	Spoiler-aware RAG	ai/rag, POST /api/book-chat, conversations	_____
8	Reading Agent	ai/agent (3 tools), POST /api/reading-agent (SSE)	_____
9	Frontend A — Reader experience	App shell, auth pages, discovery, book detail, lists, reservations, spoiler settings, chat UI	_____
10	Frontend B — Clubs, staff & agent	Clubs & meetings, moderator tools, stock management, admin roles, agent panel	_____

Why this split: each person owns a vertical slice with its own folder, routes and tests, so people rarely edit the same files. Everyone talks to each other through **shared Zod schemas** (owned by Member 1) and the API contract in Document 01 §9. Frontend developers start on mock data from day 1 and switch to the real API as each module lands.

2. Ground rules every member follows

1. **Contract first.** Before writing a route, add/agree its request and response Zod schema in shared/schemas via a small PR reviewed by Member 1.
2. **Use the shared middleware chain.** Never re-implement auth checks; use requireAuth, requireVerified, requireRole, requireClubModerator, validate(schema) from Member 2 and Member 1.
3. **Same response envelope everywhere:** res.ok(data) and throw new AppError(code, status, message) — the error handler formats it.
4. **Module layout:** model.js · service.js · controller.js · routes.js · *.test.js. Logic in services, so AI tools can call services directly.
5. **AI tools call services, never raw models,** and pass ctx.user so the same permission checks apply.
6. **Branch per task** (feat/reservations-atomic-hold), PR into dev, CI must pass, one reviewer.
7. **Write tests with your feature,** not after. Minimum: happy path + one permission failure + one validation failure per route.
8. **Never commit secrets;** add new variables to .env.example.

2.1 Code every member will use (provided first by Members 1 and 2)

```
// server/src/modules/books/routes.js – the standard pattern
import { Router } from 'express';
import { requireAuth, requireVerified, requireRole } from '../../../middleware/auth.js';
import { validate } from '../../../middleware/validate.js';
import { UpdateStockBody } from '@shared/schemas/books.js';
import * as ctrl from './controller.js';

const r = Router();
r.get('/', ctrl.list); // public
r.patch('/:id/stock', requireAuth, requireRole('bookseller'),
  validate({ body: UpdateStockBody }), ctrl.updateStock);
export default r;

// controller stays thin
export const updateStock = async (req, res) =>
  res.ok(await service.updateStock(req.params.id, req.body, req.user));
```

3. Member 1 — Architecture, contracts and integration

At a glance

Your job	Set up the repository and shared foundations so nine people can build in parallel, then bring everything together for the demo.
What you build	GitHub repo and rules, CI, shared Zod schemas and constants, app wiring (app.js, validate, AppError, error handler), seed script, end-to-end tests, demo
Your folders	shared/ · server/src/app.js · server/src/middleware/validate.js, error.js · server/src/lib/ server/scripts/seed.js · e2e/ · .github/ · docs/
Others need from you	Everyone: the repo, schemas, middleware helpers and seed data — your first three steps unblock the whole team
You need from others	Seed data files (Members 3, 5), embed step (Member 6), auth middleware for wiring (Member 2)
Your reviewer	Member 2 (any teammate can review; you review every PR that touches your folders)

Before you start

1. Collect every member's GitHub username.
2. Create a free MongoDB Atlas cluster (M0), add a database user, allow team IPs, keep the connection string private.
3. Read Document 01 Sections 8–11 fully — you own the contracts that match them.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Create the repository	
Branch	main (first commit), then dev
Build	• GitHub → New repository bookstore-clubs (private, no README) • Folder skeleton, root package.json with workspaces [shared, server, client], .gitignore, .env.example, README with setup steps • First commit and push to main; create and push dev • Settings → Collaborators: add all nine members (Write) • Settings → Default branch: dev; tick <i>Automatically delete head branches</i>
Done when	☐ Every member has cloned and run <code>npm install</code> ☐ Atlas string shared privately, never committed
STEP 2 · CI, PR template and branch protection	
Branch	chore/ci-templates
Build	• .github/workflows/ci.yml, pull_request_template.md, CODEOWNERS (code below) • Merge this through a PR — the first PR proves the setup works • Then Settings → Branches: protect main and dev — require PR, 1 approval, status check ci, code-owner review, block force push and deletion
Done when	☐ Pushing straight to dev is rejected ☐ CI runs on a test PR

STEP 3 · Shared contracts

Branch	feat/shared-contracts
Build	• shared/constants.js: roles, statuses, error codes • First-draft Zod schemas for every route in Document 01 §9 (each owner refines theirs) • docs/CONTRACT.md changelog for contract changes
Done when	☐ Server and client can import @shared/schemas/*

STEP 4 · App wiring

Branch	feat/app-wiring
Build	• app.js: helmet, JSON parser, request id + pino logger, mount every module router under /api, 404 and error handler • validate() middleware, AppError class, res.ok() helper (code below)
Done when	☐ Errors come back as { ok: false, error: { code, message } } ☐ Members can mount their routers with one line

STEP 5 · Seed script

Branch	chore/seed-script
Build	• npm run seed -- --reset wipes and recreates all demo data • Admin, booksellers, moderators and readers (verified); catalog sources; store policy docs • Wire in book/inventory data (Member 3), clubs/progress (Member 5), and the embed step (Member 6)
Done when	☐ A fresh database is demo-ready in one command ☐ Running it twice gives the same result

STEP 6 · End-to-end tests

Branch	test/e2e-demo
Build	• Playwright tests for the demo scenario (Document 01 §13) • Checks for acceptance items A1-A14 (Document 01 §14)
Done when	☐ E2E suite passes on a fresh seed

STEP 7 · Integration days and releases

Branch	PR: dev → main
Build	• Regularly: everyone pulls dev, reseeds, and walks the checklist in Section 13.3; log bugs tagged integration • When dev is stable, open a PR dev → main, merge with <i>Create a merge commit</i>, then tag a version (code below) • Before the final demo, freeze main: only bug-fix PRs
Done when	☐ Every release on main has a tag ☐ Demo always runs from the latest tag

STEP 8 · Demo and report

Branch	docs/demo-report
Build	• Demo script with timings; final report and presentation • Full rehearsal with the team on a fresh database
Done when	☐ Demo runs end-to-end in under 5 minutes

Reference code

Repository setup commands (Step 1)

```
mkdir bookstore-clubs && cd bookstore-clubs
mkdir -p shared/schemas server/src client docs .github/workflows
npm init -y # add "workspaces": ["shared","server","client"]
git init && git add . && git commit -m "chore: initial project skeleton"
git branch -M main
git remote add origin https://github.com/<you>/bookstore-clubs.git
git push -u origin main
git checkout -b dev && git push -u origin dev
```

.gitignore

```
node_modules/
.env
.env.*
!.env.example
dist/
build/
coverage/
*.log
.DS_Store
playwright-report/
```

.github/workflows/ci.yml

```
name: ci
on:
  pull_request: { branches: [dev, main] }
  push: { branches: [dev] }
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test --workspace server # mongodb-memory-server, no Atlas needed
      - run: npm run build --workspace client
```

.github/pull_request_template.md and CODEOWNERS

```
## What does this PR do?
## Member / module: Member __ · __
## How to test
## Checklist
- [ ] Up to date with dev - [ ] Schema in shared/ (if API changed)
- [ ] Tests added - [ ] lint + tests pass - [ ] No secrets / node_modules

# CODEOWNERS
/shared/ @your-github-username
/server/src/app.js @your-github-username
/server/scripts/seed.js @your-github-username
/package.json @your-github-username
/.github/ @your-github-username
```

validate() and error handler

```
export const validate = (s) => (req, _res, next) => {
  for (const k of ['body', 'query', 'params']) if (s[k]) {
    const r = s[k].safeParse(req[k]);
    if (!r.success) return next(new AppError('VALIDATION_ERROR', 400, 'Invalid input', r.error.flatten()));
    req[k] = r.data;
  }
  next();
};
export const errorHandler = (err, req, res, _next) => {
  const status = err.status ?? 500;
  req.log.error({ err }, 'request failed');
  res.status(status).json({ ok: false, error: {
    code: err.code ?? 'INTERNAL',
    message: status === 500 ? 'Something went wrong' : err.message,
    details: err.details } });
};
```

Releasing dev → main (Step 7)

```
# after the dev → main PR is merged
git checkout main
git pull origin main
git tag -a v0.3 -m "Integration: auth, catalog, reservations, clubs"
git push origin v0.3
# return to a working version any time: git checkout v0.3
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change shared/, app.js, package.json or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	git checkout dev git pull origin dev
2	Create the branch for this step	git checkout -b chore/ci-templates
3	Write code, then commit in small pieces	git add .github git commit -m "chore: add CI workflow, PR template and CODEOWNERS"
4	Bring in teammates' latest work	git fetch origin git merge origin/dev
5	Check that everything passes	npm run lint npm test --workspace server
6	Push your branch	git push -u origin chore/ci-templates
7	Open a Pull Request on GitHub → base: dev, request Member 2	(GitHub website, or gh pr create --base dev --fill)
8	After it is merged, clean up and start the next step	git checkout dev git pull origin dev git branch -d chore/ci-templates

Final checklist

- ☐ Everyone can clone, install, seed and run in under 10 minutes
- ☐ main and dev are protected; CI runs on every PR
- ☐ Acceptance checks A1–A14 pass with evidence
- ☐ Demo rehearsed on a fresh database

4. Member 2 — Authentication, sessions, roles and audit

At a glance

Your job	Build the security layer: sign-up, login, sessions, roles and the audit log. Everyone else's routes depend on your middleware.
Routes you build	/api/auth/* (signup, verify, login, logout, forgot, reset, me, csrf), GET/PATCH /api/users/me, PATCH /api/users/:id/staff-role
Your folders	server/src/modules/auth/ · users/ · audit/ server/src/middleware/auth.js · roles.js · csrf.js · rateLimit.js
Collections	users, tokens, sessions, auditLogs
Others need from you	requireAuth, requireVerified, requireRole, requireClubModerator, assertCan, audit.record — everyone
You need from others	App wiring, validate(), AppError (Member 1)
Your reviewer	Member 3

Before you start

1. Read Document 01: Section 4 (roles and permission matrix) and Section 11 (security requirements).
2. Finish laptop setup (Section 14.2) and make sure `npm run dev` starts the server.
3. Install packages: `npm i express-session connect-mongo csrf-csrf express-rate-limit argon2 nodemailer -w server`.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Session and access middleware	
Branch	feat/auth-session-middleware
Build	• User model: email, passwordHash, emailVerifiedAt, roles, moderatorOf, preferences, spoilerSettings • express-session + connect-mongo; cookie httpOnly, sameSite=lax, secure in production; idle 30 min, max 8 h • CSRF (csrf-csrf) + GET /api/auth/csrf that returns the token • Middleware: requireAuth, requireVerified, requireRole, requireClubModerator (code below)
Done when	☐ Other members can import the middleware ☐ A POST without the CSRF header returns 403 CSRF_INVALID ☐ Merge this first — the whole team is waiting on it
STEP 2 · Login, logout and current user	
Branch	feat/auth-login-logout
Build	• POST /auth/login: check argon2 hash, req.session.regenerate(), store userId • Same message for wrong email or wrong password • POST /auth/logout destroys the session; GET /auth/me returns the user without passwordHash • Login rate limit: 5 tries per 15 min per IP + email
Done when	☐ 6th wrong login returns 429 ☐ /auth/me works after login and returns 401 after logout

STEP 3 · Sign-up and email verification

Branch	feat/auth-signup-verify
Build	• POST /auth/signup: hash password with argon2id • Create a random token, store only its SHA-256 hash with 24 h expiry • Send the verify link with Nodemailer (Ethereal account in development) • GET /auth/verify?token= sets emailVerifiedAt and marks the token used
Done when	☐ Unverified user gets 403 EMAIL_NOT_VERIFIED on reserve/join ☐ Used or expired token is rejected

STEP 4 · Password recovery

Branch	feat/auth-password-reset
Build	• POST /auth/forgot: always replies 200 (don't reveal if email exists) • Single-use hashed token, 30 min expiry • POST /auth/reset: set new hash, destroy all the user's other sessions
Done when	☐ Old password stops working ☐ Reset link works only once

STEP 5 · Profile, preferences and spoiler settings

Branch	feat/users-preferences
Build	• GET/PATCH /users/me: name, preferences {genres, moods, maxPages}, spoilerSettings {strict, maxChapter} • Zod schema in shared/schemas/users.js (send to Member 1 for review)
Done when	☐ Invalid values return 400 VALIDATION_ERROR ☐ Member 7 can read spoilerSettings

STEP 6 · Staff roles and audit log

Branch	feat/users-staff-role
Build	• auditLogs model + audit.record(actor, action, target, before, after) • PATCH /users/:id/staff-role (admin only): grant/revoke bookseller or moderator-of-club • Reject when the admin targets themselves; audit both allowed and rejected attempts • Export assertCan(user, action, resource) for AI tools (Members 7, 8)
Done when	☐ Reader calling staff-role gets 403 and an audit entry exists ☐ Member 3 can call audit.record

STEP 7 · Security tests

Branch	test/auth-security
Build	• Full flow test: signup → verify → login → me → logout • Tests: CSRF missing, rate limit, self-role change, reset token reuse
Done when	☐ All tests pass in CI

Reference code

middleware/auth.js

```
export const requireAuth = async (req, _res, next) => {
  if (!req.session.userId) return next(new AppError('UNAUTHENTICATED', 401, 'Please log in'));
  req.user = await User.findById(req.session.userId).lean();
  if (!req.user) return next(new AppError('UNAUTHENTICATED', 401, 'Session expired'));
  next();
};

export const requireVerified = (req, _res, next) =>
  req.user.emailVerifiedAt ? next()
    : next(new AppError('EMAIL_NOT_VERIFIED', 403, 'Verify your email first'));

export const requireRole = (...roles) => (req, _res, next) =>
  roles.some(r => req.user.roles.includes(r)) || req.user.roles.includes('admin')
    ? next() : next(new AppError('FORBIDDEN', 403, 'Not allowed'));

export const requireClubModerator = (p = 'id') => (req, _res, next) =>
  req.user.roles.includes('admin') || req.user.moderatorOf.map(String).includes(req.params[p])
    ? next() : next(new AppError('FORBIDDEN', 403, 'Not a moderator of this club'));
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	git checkout dev git pull origin dev
2	Create the branch for this step	git checkout -b feat/auth-session-middleware
3	Write code, then commit in small pieces	git add server/src/middleware git commit -m "feat(auth): add session, csrf and requireAuth"
4	Bring in teammates' latest work	git fetch origin git merge origin/dev
5	Check that everything passes	npm run lint npm test --workspace server
6	Push your branch	git push -u origin feat/auth-session-middleware
7	Open a Pull Request on GitHub → base: dev, request Member 3	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	git checkout dev git pull origin dev git branch -d feat/auth-session-middleware

Final checklist

- ☐ Passwords and tokens never appear in logs or API responses
- ☐ No user can give themselves a staff role
- ☐ All 7 PRs merged into dev, CI green

5. Member 3 — Catalog and inventory API

At a glance

Your job	Build the book catalog and the stock numbers (new / used / unavailable) that the whole app trusts.
Routes you build	GET /api/books, GET /api/books/:id, POST/PATCH /api/books (bookseller), PATCH /api/books/:id/stock
Your folders	server/src/modules/books/ · inventory/ · catalogSources/ server/scripts/data/books.json · inventory.json
Collections	books, inventory, catalogSources
Others need from you	Inventory model (Member 4), books to embed (Member 6), getAvailability(bookIds) (Member 8), APIs (Members 9, 10)
You need from others	requireRole, audit.record (Member 2)
Your reviewer	Member 4

Before you start

1. Read Document 01: Section 7 (M3), Section 8 (books, inventory, catalogSources) and Section 9 (routes).
2. Finish laptop setup (Section 14.2).
3. Until Member 2's middleware is merged, write your public GET routes first — they need no login.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Models and basic listing	
Branch	feat/books-model-list
Build	• Book model: title, authors, isbn, themes, moods, synopsis, pageCount, chapterCount, readingHours, sourceId, contentLevel, approvedSource, embedding • Inventory model: bookId, condition (new/used), total, available, held, status, pickupWindows • CatalogSource model: name, type, license, permission • GET /books and GET /books/:id — never return the embedding field
Done when	☐ Seeded books appear at GET /api/books ☐ Unique index on (bookId, condition)
STEP 2 · Filters and availability	
Branch	feat/books-filters-availability
Build	• Query params: q, theme, mood, maxPages, available, page, limit • Add availability summary per book, e.g. { new: 3, used: 0, reservable: true } • Unavailable or 0 copies → reservable: false with a clear label • List only books whose source is approved
Done when	☐ Each filter tested ☐ Unavailable titles are clearly not reservable

STEP 3 · Bookseller book editing

Branch	feat/books-bookseller-crud
Build	• POST/PATCH /books for booksellers (requireRole('bookseller')) • After synopsis or themes change, call <code>embeddings.queueBook(id)</code> (Member 6)
Done when	☐ Reader gets 403 ☐ Changes audited

STEP 4 · Stock changes

Branch	feat/inventory-stock-adjust
Build	• PATCH /books/:id/stock: adjust counts, set unavailable • Stock can never go negative (atomic update, code below) • Every change calls <code>audit.record</code> with before/after
Done when	☐ Negative stock returns 409 ☐ Audit entry exists for each change

STEP 5 · Bulk availability for the agent

Branch	feat/inventory-bulk-availability
Build	• Export <code>getAvailability(bookIds)</code> — one \$in query, returns a map by bookId
Done when	☐ Member 8 can call it with a list of ids

STEP 6 · Demo seed data

Branch	chore/books-seed-data
Build	• About 40 books with themes, moods and synopses from permitted sources • Include the demo pair: a popular mystery with only 3 copies, and a short mystery (~220 pages) with 10 copies • Give the files to Member 1 to wire into the seed script
Done when	☐ <code>npm run seed</code> loads your data

Reference code

inventory/service.js — stock never goes negative

```
export async function adjust(bookId, condition, delta, actor) {
  const before = await Inventory.findOne({ bookId, condition }).lean();
  const after = await Inventory.findOneAndUpdate(
    { bookId, condition, available: { $gte: -Math.min(delta, 0) } },
    { $inc: { total: delta, available: delta } }, { new: true });
  if (!after) throw new AppError('CONFLICT', 409, 'Stock would go negative');
  await audit.record(actor, 'stock.adjust', { type: 'inventory', id: after._id }, before, after);
  return after;
}
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	<pre>git checkout dev git pull origin dev</pre>
2	Create the branch for this step	<pre>git checkout -b feat/books-model-list</pre>

#	Do this	Command
3	Write code, then commit in small pieces	git add server/src/modules/books git commit -m "feat(books): add book model and GET /api/books"
4	Bring in teammates' latest work	git fetch origin git merge origin/dev
5	Check that everything passes	npm run lint npm test --workspace server
6	Push your branch	git push -u origin feat/books-model-list
7	Open a Pull Request on GitHub → base: dev, request Member 4	(GitHub website, or gh pr create --base dev --fill)
8	After it is merged, clean up and start the next step	git checkout dev git pull origin dev git branch -d feat/books-model-list

Final checklist

- ☐ List responses never include embeddings
- ☐ Every stock change is audited
- ☐ All 6 PRs merged, CI green

6. Member 4 — Reservations and reading lists API

At a glance

Your job	Let readers reserve copies correctly — no overselling when many people click at once, and abandoned holds expire. Also build private reading lists.
Routes you build	POST /api/books/:id/reservations, GET /api/reservations/mine, DELETE /api/reservations/:id, POST /api/reservations/:id/collection, /api/reading-lists
Your folders	server/src/modules/reservations/ · readingLists/ server/src/jobs/expireHolds.js
Collections	reservations, readingLists
Others need from you	Reservation and list APIs (Members 9, 10), race and expiry tests for acceptance
You need from others	Inventory model and pickup windows (Member 3), requireVerified / requireRole (Member 2)
Your reviewer	Member 5

Before you start

1. Read Document 01: Section 7 (M4, M5), Section 8 (reservations, readingLists) and Section 14 (A1, A2, A14).
2. Finish laptop setup (Section 14.2).
3. Read about MongoDB transactions; tests need MongoMemoryReplSet (transactions need a replica set).

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Reading lists	
Branch	feat/reading-lists-crud
Build	• ReadingList model: ownerId, name, visibility (private by default), items • Create, read, rename, add/remove book, delete • Every query filters by ownerId: req.user._id
Done when	☐ User B gets 404 for user A's list
STEP 2 · Reserve a copy (atomic)	
Branch	feat/reservations-atomic-hold
Build	• Reservation model: userId, bookId, condition, status, pickupWindow, expiresAt • In a transaction: decrease available only if ≥ 1, increase held, create the reservation (code below) • No copy left → 409 OUT_OF_STOCK
Done when	☐ Verified readers only ☐ Stock numbers stay correct

STEP 3 · My reservations and cancel

Branch	feat/reservations-mine-cancel
Build	• GET /reservations/mine for the logged-in reader • DELETE /reservations/:id — owner only, releases the copy exactly once
Done when	☐ Cancelling twice does not add stock twice

STEP 4 · Collection by bookseller

Branch	feat/reservations-collection
Build	• POST /reservations/:id/collection: held → collected, reduce held and total
Done when	☐ Readers get 403

STEP 5 · Expire abandoned holds

Branch	feat/jobs-expire-holds
Build	• node-cron job every minute: held + past expiresAt → expired, release stock • Must be safe to run twice (status check in the update filter)
Done when	☐ Expired hold gives the copy back

STEP 6 · Race and expiry tests

Branch	test/reservations-race-expiry
Build	• 20 parallel reserve requests for a title with 3 copies • Hold with a past expiresAt + run the job • Agent run creates zero reservations (with Member 8)
Done when	☐ Exactly 3 succeed and 17 get 409 ☐ total = available + held after every test

Reference code

reservations/service.js — atomic reserve

```
export async function reserve(user, bookId, { condition, pickupWindowId }) {
  const session = await mongoose.startSession();
  try {
    return await session.withTransaction(async () => {
      const inv = await Inventory.findOneAndUpdate(
        { bookId, condition, status: 'active', available: { $gte: 1 } },
        { $inc: { available: -1, held: 1 } }, { new: true, session });
      if (!inv) throw new AppError('OUT_OF_STOCK', 409, 'No copy available');
      const win = inv.pickupWindows.id(pickupWindowId);
      if (!win) throw new AppError('VALIDATION_ERROR', 400, 'Unknown pickup window');
      const [r] = await Reservation.create([
        { userId: user._id, bookId, condition, status: 'held', pickupWindow: win, expiresAt: win.end }
      ], { session });
      return r;
    });
  } finally { session.endSession(); }
}
```

jobs/expireHolds.js — safe to run twice

```
const due = await Reservation.find({ status: 'held', expiresAt: { $lt: new Date() } });
for (const r of due) {
  const upd = await Reservation.updateOne({ _id: r._id, status: 'held' }, { status: 'expired' });
  if (upd.modifiedCount) await Inventory.updateOne(
    { bookId: r.bookId, condition: r.condition }, { $inc: { available: 1, held: -1 } });
}
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	<code>git checkout dev</code> <code>git pull origin dev</code>
2	Create the branch for this step	<code>git checkout -b feat/reading-lists-crud</code>
3	Write code, then commit in small pieces	<code>git add server/src/modules/readingLists</code> <code>git commit -m "feat(reading-lists): add CRUD for owner"</code>
4	Bring in teammates' latest work	<code>git fetch origin</code> <code>git merge origin/dev</code>
5	Check that everything passes	<code>npm run lint</code> <code>npm test --workspace server</code>
6	Push your branch	<code>git push -u origin feat/reading-lists-crud</code>
7	Open a Pull Request on GitHub → base: dev, request Member 5	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d feat/reading-lists-crud</code>

Final checklist

- ☐ Race test (A1) and expiry test (A2) pass in CI
- ☐ Private data is owner-only everywhere
- ☐ All 6 PRs merged, CI green

7. Member 5 — Clubs, meetings and reading progress API

At a glance

Your job	Build reading clubs, meetings and members, and track how far each reader has read — this decides what counts as a spoiler.
Routes you build	GET /api/clubs, POST /api/clubs/:id/join, POST /api/clubs/:id/meetings, POST /api/clubs/:id/excerpts/:eid/approve, PATCH /api/reading-progress
Your folders	server/src/modules/clubs/ · memberships/ · meetings/ · progress/ server/scripts/data/clubs.json
Collections	clubs, memberships, meetings, progress (and approval fields on excerpts)
Others need from you	getBoundary() (Member 7), getConstraints() (Member 8), club APIs (Members 9, 10)
You need from others	requireClubModerator, assertCan (Member 2); excerpts with chapter data (Member 6)
Your reviewer	Member 2

Before you start

1. Read Document 01: Section 4 (moderator role), Section 7 (M5, M6), Section 10.2 (spoiler boundary).
2. Finish laptop setup (Section 14.2).
3. Agree the shape of getBoundary() with Member 7 and getConstraints() with Member 8 on day one.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Clubs list and joining	
Branch	feat/clubs-list-join
Build	• Club model: name, description, published, rules, moderators, currentBookId, readingPace.pagesPerWeek • Membership model with unique (clubId, userId) • GET /clubs: published clubs with next meeting date; POST /clubs/:id/join for verified readers
Done when	☐ Joining twice returns 409
STEP 2 · Meetings and members	
Branch	feat/clubs-meetings
Build	• Meeting model: clubId, date, bookId, agenda, chapterRange • Moderator of that club only: create meetings, remove members, edit rules, set current book
Done when	☐ Moderator of club X gets 403 on club Y

STEP 3 · Reading progress**Branch** feat/progress-api**Build**

- PATCH /reading-progress: chapter, page, visibility (private / club / public)
- GET /reading-progress/mine; members see others' progress only if shared

Done when ☐ Private progress never visible to others**STEP 4 · Spoiler boundary****Branch** feat/progress-boundary**Build**

- Export getBoundary(userId, bookId, clubId?) = the smaller of declared chapter and the user's spoiler limit (code below)

Done when ☐ Unit tests: no progress, strict limit, club vs personal**STEP 5 · Approve discussion excerpts****Branch** feat/excerpts-approval**Build**

- POST /clubs/:id/excerpts/:eid/approve sets approved and approvedBy
- Excerpts without chapter data cannot be approved

Done when ☐ Only that club's moderator can approve**STEP 6 · Club constraints for the agent****Branch** feat/clubs-constraints**Build**

- Export getConstraints(clubId, user): members, next meeting, days left, pages per week

Done when ☐ Member 8 can call it**STEP 7 · Demo seed data****Branch** chore/clubs-seed-data**Build**

- Two clubs, including "Tuesday Mystery Circle" with 8 members and a meeting in 14 days
- Progress records for the demo readers

Done when ☐ npm run seed loads your data

Reference code

progress and clubs services

```

export async function getBoundary(userId, bookId, clubId) {
  const p = await Progress.findOne({ userId, bookId, ...(clubId && { clubId }) }).lean();
  const u = await User.findById(userId, 'spoilerSettings').lean();
  const declared = p?.chapter ?? 0; // nothing declared → no spoilers
  const limit = u.spoilerSettings?.strict ? u.spoilerSettings.maxChapter ?? declared : declared;
  return { chapter: Math.min(declared, limit), page: p?.page ?? 0, declared: !!p };
}

export async function getConstraints(clubId, user) {
  assertCan(user, 'club.read', { clubId });
  const club = await Club.findById(clubId).lean();
  const next = await Meeting.findOne({ clubId, date: { $gte: new Date() } }).sort('date').lean();
  const members = await Membership.countDocuments({ clubId, status: 'active' });
  const days = next ? Math.ceil((next.date - Date.now()) / 864e5) : null;
  return { members, nextMeeting: next?.date, days, pagesPerWeek: club.readingPace.pagesPerWeek };
}

```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	<code>git checkout dev</code> <code>git pull origin dev</code>
2	Create the branch for this step	<code>git checkout -b feat/clubs-list-join</code>
3	Write code, then commit in small pieces	<code>git add server/src/modules/clubs</code> <code>git commit -m "feat(clubs): list published clubs and join"</code>
4	Bring in teammates' latest work	<code>git fetch origin</code> <code>git merge origin/dev</code>
5	Check that everything passes	<code>npm run lint</code> <code>npm test --workspace server</code>
6	Push your branch	<code>git push -u origin feat/clubs-list-join</code>
7	Open a Pull Request on GitHub → base: dev, request Member 2	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d feat/clubs-list-join</code>

Final checklist

- ☐ Moderator rights are checked per club
- ☐ `getBoundary` fully tested
- ☐ All 7 PRs merged, CI green

8. Member 6 — AI foundations and semantic discovery

At a glance

Your job	Set up the local AI (Ollama), turn text into vectors, and let readers find books by describing what they remember.
What you build	Ollama client, embeddings, Atlas Vector Search indexes, ingestion of excerpts and store documents, the discovery endpoint
Your folders	server/src/ai/ollama.js · embeddings/ · ingest/ · discovery/docs/indexes/
Others need from you	Retrievers and LLM client (Member 7), discover() (Member 8), discovery endpoint (Member 9), embed step in seed (Member 1)
You need from others	Books and getAvailability (Member 3), excerpts with chapter data (Member 5)
Your reviewer	Member 7

Before you start

1. Read Document 01: Section 10.1 (semantic discovery) and Section 8 (books, excerpts, storeDocs).
2. Install Ollama, then run `ollama pull nomic-embed-text` and `ollama pull llama3.1:8b`.
3. Install: `npm i @langchain/ollama @langchain/core langchain -w server`.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Ollama client and setup guide	
Branch	chore/ai-ollama-client
Build	• ai/ollama.js: shared ChatOllama and OllamaEmbeddings with timeouts • Health check: if Ollama is down, routes return 503 AI_UNAVAILABLE • Ollama setup steps in the README for the team
Done when	☐ Stopping Ollama gives a friendly error, not a crash

STEP 2 · Book embeddings	
Branch	feat/ai-book-embeddings
Build	• Embed title + themes + moods + synopsis into books.embedding (768 numbers) • Create the books_vec index in Atlas; save its JSON in docs/indexes/
Done when	☐ All seeded books have embeddings

STEP 3 · Excerpts and store documents	
Branch	feat/ai-ingest-excerpts-docs
Build	• Ingest excerpts, keeping bookId, chapter, page, approved • Split store documents into ~500-token chunks with 50 overlap • Create excerpts_vec (filters: bookId, approved, chapter) and storedocs_vec
Done when	☐ Member 7 can search both

STEP 4 · Embedding in the seed	
Branch	feat/ai-embed-seed-step
Build	• npm run embed script used by the seed • queueBook(id) hook that Member 3 calls when a book changes
Done when	☐ A fresh seed produces searchable data

STEP 5 · Discovery endpoint	
Branch	feat/ai-discovery
Build	• Embed the query, run \$vectorSearch on approved books, add availability • Ask the LLM for a 1–2 sentence "why it matched" using only the stored themes, moods and synopsis
Done when	☐ Explanations never invent plot details

STEP 6 · Evaluation set	
Branch	test/ai-discovery-eval
Build	• 10 "what I remember" queries with the expected book • Report how many hit the top 5
Done when	☐ At least 7 of 10 found

Reference code

Atlas index definition: excerpts_vec

```
{ "fields": [
  { "type": "vector", "path": "embedding", "numDimensions": 768, "similarity": "cosine" },
  { "type": "filter", "path": "bookId" },
  { "type": "filter", "path": "approved" },
  { "type": "filter", "path": "chapter" } ] }
```

ai/discovery/service.js

```
export async function discover(query) {
  const v = await embeddings.embedQuery(query);
  const hits = await Book.aggregate([
    { $vectorSearch: { index: 'books_vec', path: 'embedding', queryVector: v,
      numCandidates: 100, limit: 8, filter: { approvedSource: true } } },
    { $project: { embedding: 0, score: { $meta: 'vectorSearchScore' } } } ]]);
  const avail = await inventoryService.getAvailability(hits.map(h => h._id));
  const why = await explainMatches(query, hits); // prompt: use ONLY the given fields
  return hits.map(h => ({ ...h, availability: avail[h._id], why: why[h._id] }));
}
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change shared/, app.js, package.json or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	git checkout dev git pull origin dev
2	Create the branch for this step	git checkout -b chore/ai-ollama-client
3	Write code, then commit in small pieces	git add server/src/ai/ollama.js git commit -m "chore(ai): add ollama client and health check"

#	Do this	Command
4	Bring in teammates' latest work	<code>git fetch origin</code> <code>git merge origin/dev</code>
5	Check that everything passes	<code>npm run lint</code> <code>npm test --workspace server</code>
6	Push your branch	<code>git push -u origin chore/ai-ollama-client</code>
7	Open a Pull Request on GitHub → base: dev, request Member 7	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d chore/ai-ollama-client</code>

Final checklist

- ☐ No unapproved source is ever returned
- ☐ AI_UNAVAILABLE handled
- ☐ All 6 PRs merged, CI green

9. Member 7 — Spoiler-aware RAG (book chat)

At a glance

Your job	Build the book chat: it answers store-policy questions and discusses approved excerpts, but only up to where the reader has read — with citations, and honestly when it doesn't know.
Route you build	POST /api/book-chat { message, bookId?, clubId?, conversationId? }
Your folders	server/src/ai/rag/ · server/src/modules/conversations/
Others need from you	Chat response { answer, citations[], status } (Member 9)
You need from others	getBoundary, approved excerpts (Member 5); retrievers, LLM (Member 6); assertCan, rate limit (Member 2)
Your reviewer	Member 8

Before you start

1. Read Document 01: Section 10.2 (spoiler-aware RAG) and Section 14 (A4–A7).
2. Read the LangChain JS retrieval (RAG) tutorial.
3. Until Members 5 and 6 merge, use a mocked retriever and boundary.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Prompts and mock retriever	
Branch	feat/rag-prompts-mock
Build	• ai/rag/prompts.js with the four rules (box below) • A mock retriever returning fixed chunks so you can build early
Done when	☐ Prompt reviewed by the AI trio
STEP 2 · Store-policy questions	
Branch	feat/rag-policy-qa
Build	• Detect policy vs book questions • Answer policy questions from storeDocs with citations
Done when	☐ "How long is a hold?" answered with a cited policy
STEP 3 · Spoiler filter	
Branch	feat/rag-spoiler-filter
Build	• Get the reader's boundary; if none, ask "How far have you read?" • Search excerpts with { bookId, approved: true, chapter: { \$lte: boundary } } • Club questions also require membership
Done when	☐ No result beyond the boundary is ever retrieved

STEP 4 · Safety checks and missing data

Branch	feat/rag-postcheck-missing-data
Build	• Drop any citation not retrieved or past the boundary • Synopsis-only books: say so and decline plot questions • Nothing relevant found: graceful "I don't have approved material on that"
Done when	☐ Unsupported plot questions are declined honestly

STEP 5 · Save conversations

Branch	feat/rag-conversations
Build	• Store messages with citations; owner-only
Done when	☐ Another user cannot read them

STEP 6 · Spoiler test set

Branch	test/rag-spoiler-set
Build	• At least 15 questions: within boundary, beyond boundary, synopsis-only, no metadata, policy
Done when	☐ Zero spoiler leaks

Reference code

ai/rag/bookChat.js — outline

```
export async function bookChat(user, { message, bookId, clubId }) {
  if (clubId) assertCan(user, 'club.member', { clubId });
  if (await classify(message) === 'policy') return answerFrom(await storeDocRetriever(message), message);

  const book = await bookService.get(bookId);
  const b = await progressService.getBoundary(user._id, bookId, clubId);
  if (!b.declared) return ask('How far have you read? Tell me a chapter so I avoid spoilers.');
```

```
  if (book.contentLevel === 'synopsis') return synopsisOnly(book, message);

  const chunks = await excerptRetriever(message, { bookId, approved: true, chapter: { $lte: b.chapter } });
  if (!chunks.length || chunks[0].score < 0.6)
    return missing("I don't have approved material up to your chapter that covers this.");
  const out = await llm.invoke(ragPrompt({ chunks, message, boundary: b.chapter }));
  return withVerifiedCitations(out, chunks, b.chapter);
}
```

Prompt rules (ai/rag/prompts.js)

1. Answer only from CONTEXT. If CONTEXT does not contain the answer, say you don't have it.
2. Never describe events after chapter {boundary}. Never guess endings.
3. Cite every claim as [n] using the context ids.
4. If you only have a synopsis, say so plainly.

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change shared/, app.js, package.json or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	git checkout dev git pull origin dev
2	Create the branch for this step	git checkout -b feat/rag-prompts-mock
3	Write code, then commit in small pieces	git add server/src/ai/rag git commit -m "feat(rag): add prompts and mock retriever"

#	Do this	Command
4	Bring in teammates' latest work	<code>git fetch origin</code> <code>git merge origin/dev</code>
5	Check that everything passes	<code>npm run lint</code> <code>npm test --workspace server</code>
6	Push your branch	<code>git push -u origin feat/rag-prompts-mock</code>
7	Open a Pull Request on GitHub → base: dev, request Member 8	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d feat/rag-prompts-mock</code>

Final checklist

- ☐ Every sourced answer shows citations
- ☐ AI_UNAVAILABLE handled
- ☐ All 6 PRs merged, CI green

10. Member 8 — Reading agent workflow

At a glance

Your job	Build the agent that helps a club pick its next book: search → check copies and length → compare with the club's schedule → shortlist with evidence.
Route you build	POST /api/reading-agent { clubId, request } → streams tool status (Server-Sent Events)
Your folders	server/src/ai/agent/ (tools/, agent.js, events.js)
Others need from you	SSE event format (Member 10), demo scenario (Member 1)
You need from others	discover() (Member 6), getAvailability + book lengths (Member 3), getConstraints (Member 5), assertCan (Member 2)
Your reviewer	Member 6

Before you start

1. Read Document 01: Section 10.3 (agent workflow) and Section 13 (demo scenario).
2. Read the LangChain JS tool-calling agent tutorial.
3. Agree the SSE event format with Member 10 on day one: { name, status, summary } and a final { shortlist }.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · The three tools (mocked data)

Branch	feat/agent-tools
Build	• searchCatalog(preferences) → candidate books with match evidence • checkStockAndLength(bookIds, groupSize) → copies, pages, enoughCopies • compareClubConstraints(clubId, books) → time-fit and content-fit notes • Each tool has a Zod input schema and emits started / succeeded / failed
Done when	☐ Unit tests for each tool with mocked services

STEP 2 · Connect to real services

Branch	feat/agent-real-services
Build	• Replace mocks with Members 3, 5, 6 services • Every tool passes the caller and re-checks permissions
Done when	☐ Non-member gets 403 from compareClubConstraints

STEP 3 · Streaming route

Branch	feat/agent-sse-route
Build	• POST /reading-agent streams tool events, then the final shortlist (max 3) with evidence
Done when	☐ Events visible with curl

STEP 4 · Fallback chain	
Branch	feat/agent-chain-fallback
Build	• If the local model is weak at tool calling, run the 3 tools in fixed order (AGENT_MODE=chain) • Temperature 0 so the demo is repeatable
Done when	☐ Both modes give the same demo result

STEP 5 · Demo scenario test	
Branch	test/agent-demo-scenario
Build	• Popular mystery flagged "not enough copies"; short alternative recommended with time-fit • An agent run creates zero reservations
Done when	☐ Test passes on fresh seed data

Reference code

tools/checkStockAndLength.js

```
export const checkStockAndLength = (ctx) => tool(async ({ bookIds, groupSize }) => {
  ctx.emit('tool', { name: 'checkStockAndLength', status: 'started' });
  const avail = await inventoryService.getAvailability(bookIds);
  const books = await bookService.getMany(bookIds, 'title pageCount readingHours');
  const res = books.map(b => {
    const copies = (avail[b._id]?.new ?? 0) + (avail[b._id]?.used ?? 0);
    return { id: b._id, title: b.title, pages: b.pageCount, copies, enoughCopies: copies >= groupSize };
  });
  ctx.emit('tool', { name: 'checkStockAndLength', status: 'succeeded',
    summary: `${res.filter(r => r.enoughCopies).length}/${res.length} have enough copies` });
  return JSON.stringify(res);
}, { name: 'checkStockAndLength',
  description: 'Check available copies and length for candidate books',
  schema: z.object({ bookIds: z.array(z.string()).max(10), groupSize: z.number().int().min(1) }) });
```

SSE in the route

```
res.writeHead(200, { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-cache' });
const emit = (type, data) => res.write(`event: ${type}\ndata: ${JSON.stringify(data)}\n\n`);
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change shared/, app.js, package.json or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	git checkout dev git pull origin dev
2	Create the branch for this step	git checkout -b feat/agent-tools
3	Write code, then commit in small pieces	git add server/src/ai/agent git commit -m "feat(agent): add three tools with zod schemas"
4	Bring in teammates' latest work	git fetch origin git merge origin/dev
5	Check that everything passes	npm run lint npm test --workspace server
6	Push your branch	git push -u origin feat/agent-tools
7	Open a Pull Request on GitHub → base: dev, request Member 6	(GitHub website, or gh pr create --base dev --fill)

#	Do this	Command
8	After it is merged, clean up and start the next step	<pre>git checkout dev git pull origin dev git branch -d feat/agent-tools</pre>

Final checklist

- ☐ The agent never reserves or selects a book
- ☐ Failed tools show a failed status and a friendly message
- ☐ All 5 PRs merged, CI green

11. Member 9 — Frontend A: reader experience

At a glance

Your job	Build the app shell and every screen a reader uses: sign-in, discovery, book pages, reservations, reading lists, spoiler settings and book chat.
Screens you build	Login, Signup, Verify, Forgot/Reset, Discovery (home), Book detail, Reading lists, My reservations, Progress & spoiler settings, Book chat
Your folders	client/src/app/ · api/ · components/ client/src/features/auth · discovery · books · readingLists · reservations · settings · chat
Others need from you	App shell, api client and shared components (Member 10)
You need from others	Auth APIs (2), catalog (3), reservations and lists (4), progress (5), discovery (6), book chat (7)
Your reviewer	Member 10

Before you start

1. Read Document 01: Section 9 (API) and Section 12 (usability, accessibility).
2. Agree colours, fonts and basic components with Member 10.
3. Start with mock data (MSW) and switch each screen to the real API when that backend PR is merged.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · App shell and api client	
Branch	feat/ui-shell-api-client
Build	• Vite + React + React Router + TanStack Query • Layout with navigation that changes by role (from /auth/me) • api/client.js with cookies and the CSRF header (code below) • Components: Button, Card, Badge, Dialog, Input, Spinner, ErrorState
Done when	☐ Merge early — Member 10 builds on it
STEP 2 · Sign-in screens	
Branch	feat/ui-auth-pages
Build	• Login, Signup, Verify-email page, Forgot and Reset password • "Please verify your email" banner; protected route wrapper
Done when	☐ Full sign-up → login works on the real API

STEP 3 · Catalog and book detail

Branch	feat/ui-catalog-book-detail
Build	• Catalog list with filters • Book detail with availability per condition • Unavailable looks clearly different from Reservable; Reserve dialog with pickup window and confirm
Done when	☐ Unavailable books have no Reserve button ☐ 409 shows "no longer available"

STEP 4 · Reservations and reading lists

Branch	feat/ui-reservations-lists
Build	• My reservations: status, time left, cancel • Reading lists: create, rename, add/remove books
Done when	☐ Cancel updates the list immediately

STEP 5 · Progress and spoiler settings

Branch	feat/ui-progress-spoiler-settings
Build	• Set chapter/page per book and visibility • Strict spoiler limit option
Done when	☐ Saved values reload correctly

STEP 6 · Discovery

Branch	feat/ui-discovery
Build	• Search box: "Describe what you remember" • Result cards with "why it matched", theme chips, availability badge
Done when	☐ Loading, empty and AI-unavailable states

STEP 7 · Book chat

Branch	feat/ui-book-chat
Build	• Chat panel with message list • Citation chips (source, chapter); clarifying-question, missing-data and AI-unavailable states
Done when	☐ Citations visible on every sourced answer

Reference code

api/client.js

```
const api = axios.create({ baseURL: '/api', withCredentials: true });
let csrf;
api.interceptors.request.use(async (cfg) => {
  if (cfg.method !== 'get') {
    csrf ??= (await axios.get('/api/auth/csrf', { withCredentials: true })).data.data.token;
    cfg.headers['x-csrf-token'] = csrf;
  }
  return cfg;
});
api.interceptors.response.use(r => r.data.data,
  e => Promise.reject(e.response?.data?.error ?? { code: 'NETWORK', message: 'Network error' }));
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	<code>git checkout dev</code> <code>git pull origin dev</code>
2	Create the branch for this step	<code>git checkout -b feat/ui-shell-api-client</code>
3	Write code, then commit in small pieces	<code>git add client/src/app</code> <code>git commit -m "feat(ui): add app shell and api client"</code>
4	Bring in teammates' latest work	<code>git fetch origin</code> <code>git merge origin/dev</code>
5	Check that everything passes	<code>npm run lint</code> <code>npm test --workspace client</code>
6	Push your branch	<code>git push -u origin feat/ui-shell-api-client</code>
7	Open a Pull Request on GitHub → base: dev, request Member 10	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d feat/ui-shell-api-client</code>

Final checklist

- ☐ Every screen has loading, empty and error states
- ☐ Inputs have labels; works on phone width
- ☐ Add data-testid on main buttons for end-to-end tests
- ☐ All 7 PRs merged, CI green

12. Member 10 — Frontend B: clubs, staff and agent screens

At a glance

Your job	Build the screens for clubs and moderators, booksellers and the admin, plus the live agent panel.
Screens you build	Clubs list and detail, Moderator console, Stock management, Collection confirmation, Admin staff roles, Agent panel
Your folders	client/src/features/clubs · meetings · moderator · stock · admin · agent
Others need from you	Shared components you add go back to Member 9's kit
You need from others	Shell and api client (9), clubs (5), stock and collection (3, 4), staff roles (2), agent stream (8)
Your reviewer	Member 9

Before you start

1. Read Document 01: Section 4 (roles), Section 9 (API) and Section 13 (demo scenario).
2. Agree colours, fonts and components with Member 9; wait for their shell PR, then branch from dev.
3. Agree the agent event format with Member 8.

Your tasks, step by step

Do the steps **in order**. Each step is **one branch and one Pull Request**. Tick the *Done when* boxes before you open the PR.

STEP 1 · Clubs	
Branch	feat/ui-clubs
Build	• Clubs list (public) and club detail: description, rules, meetings, current book • Join button for verified readers
Done when	☐ Joining works on the real API
STEP 2 · Stock management	
Branch	feat/ui-stock-management
Build	• Table of books with new/used counts; adjust with a reason; mark unavailable • Reservation queue with "Confirm collection"
Done when	☐ Visible to booksellers only (server also blocks others)
STEP 3 · Moderator console	
Branch	feat/ui-moderator-console
Build	• Create meetings, manage members, approve excerpts, set current book
Done when	☐ Shown only for clubs the user moderates

STEP 4 · Admin staff roles

Branch	feat/ui-admin-roles
Build	• Search a user; grant/revoke bookseller or moderator-of-club • Disabled for your own account
Done when	☐ New moderator sees the console after refresh

STEP 5 · Agent panel

Branch	feat/ui-agent-panel
Build	• Request box, live tool timeline (started → succeeded / failed) • Final shortlist cards with evidence and citations • "Select for club" (moderator) and a Reserve link for readers
Done when	☐ Demo scenario runs end-to-end in the UI

Reference code

features/agent/useAgentStream.js

```
export function useAgentStream() {
  const [events, setEvents] = useState([]); const [result, setResult] = useState(null);
  async function run(clubId, request) {
    const res = await fetch('/api/reading-agent', { method: 'POST', credentials: 'include',
      headers: { 'Content-Type': 'application/json', 'x-csrf-token': await getCsrf() },
      body: JSON.stringify({ clubId, request }) });
    const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
    let buf = '';
    for (;;) {
      const { value, done } = await reader.read(); if (done) break;
      buf += value; const parts = buf.split('\n\n'); buf = parts.pop();
      for (const p of parts) {
        const [, type] = p.match(/event: (.*)/) ?? [];
        const data = JSON.parse(p.split('data: ')[1]);
        type === 'final' ? setResult(data) : setEvents(e => [...e, data]);
      }
    }
  }
  return { events, result, run };
}
```

Git: repeat this for every step

The example uses your Step 1 branch. For later steps, use that step's branch name. Only commit inside your own folders. If you need to change `shared/`, `app.js`, `package.json` or the seed script, make a small separate PR and request Member 1 as well. Full Git guide and fixes: Section 14.

#	Do this	Command
1	Get the latest dev	<pre>git checkout dev git pull origin dev</pre>
2	Create the branch for this step	<pre>git checkout -b feat/ui-clubs</pre>
3	Write code, then commit in small pieces	<pre>git add client/src/features/clubs git commit -m "feat(ui): clubs list and detail"</pre>
4	Bring in teammates' latest work	<pre>git fetch origin git merge origin/dev</pre>
5	Check that everything passes	<pre>npm run lint npm test --workspace client</pre>
6	Push your branch	<pre>git push -u origin feat/ui-clubs</pre>

#	Do this	Command
7	Open a Pull Request on GitHub → base: dev, request Member 9	(GitHub website, or <code>gh pr create --base dev --fill</code>)
8	After it is merged, clean up and start the next step	<code>git checkout dev</code> <code>git pull origin dev</code> <code>git branch -d feat/ui-clubs</code>

Final checklist

- ☐ UI never shows actions a role can't do
- ☐ Every screen has loading, empty and error states
- ☐ All 5 PRs merged, CI green

13. Integration plan

13.1 The three integration seams

Seam	Contract	Guarded by
Frontend ↔ API	Routes in Doc 01 §9, request/response Zod schemas in shared/, envelope {ok, data error}	Schema tests; MSW mocks generated from the same schemas
Module ↔ module (backend)	Exported service functions (e.g. getAvailability, getBoundary, getConstraints, discover, assertCan)	Unit tests by provider; consumers mock until real one lands
AI ↔ application	Tools call services with ctx.user; SSE event format {type, name, status, summary}; chat response {answer, citations[], status}	Agent/RAG tests on seeded data

13.2 How we integrate, step by step

1. **Skeleton end-to-end first:** Member 1 merges repo + app wiring; Member 2 merges login; Member 3 merges GET /books; Member 9 shows books on the home page from the real API. The whole pipe works before features grow.
2. **Mock first, swap later:** frontend uses MSW handlers per module. When a backend PR merges, the frontend owner deletes that handler in a follow-up PR — one module at a time.
3. **Integration days** (held regularly, e.g. after each major module lands): everyone pulls dev, runs `npm run seed -- --reset`, and walks the checklist below. Bugs go on the board tagged integration.
4. **Contract changes** only via PR to shared/ reviewed by Member 1 and announced in the group with what breaks.
5. **Seed is the shared truth:** every demo and test uses the same seed data, so "works on my machine" problems disappear.
6. **Freeze** before the final demo: only bug fixes merge to main.

13.3 Integration checklist (run on each integration day)

Flow	Members involved	Expected
Signup → verify → login → me	2, 9	Session cookie set, role-aware nav
Browse & discover → book detail	3, 6, 9	Results with why-matched and availability
Reserve → my reservations → cancel	3, 4, 9	Stock numbers update; 409 shown nicely
Bookseller adjust stock → confirm collection	3, 4, 10	Audit entry exists
Join club → set progress → moderator meeting	5, 9, 10	Permissions respected
Book chat within boundary	5, 6, 7, 9	Cited answer, no spoilers
Agent demo scenario	3, 5, 6, 8, 10	Live tool status and correct shortlist
Admin grants moderator role	2, 10	New moderator sees console
Ollama stopped	6, 7, 8, 9, 10	Graceful AI_UNAVAILABLE everywhere

13.4 Environment variables (.env.example)

```
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster0.xxxx.mongodb.net/bookstore
SESSION_SECRET=change-me
CSRF_SECRET=change-me
CLIENT_ORIGIN=http://localhost:5173
SMTP_HOST=smtp.ethereal.email SMTP_USER= SMTP_PASS=
OLLAMA_URL=http://localhost:11434
OLLAMA_CHAT_MODEL=llama3.1:8b
OLLAMA_EMBED_MODEL=nomic-embed-text
AGENT_MODE=tools          # or 'chain' (deterministic fallback)
```

14. Git workflow — step by step

All integration of code happens through **one GitHub repository**. Nobody emails zip files or pastes code in chat. The repository, branch protection, CI and templates are already set up; every change reaches the project through a branch and a Pull Request (PR), which is checked automatically and reviewed by a teammate.

```
main ← stable, demo-ready code only (protected, merged from dev after integration days)
├─ dev ← everyone's finished features meet here and get tested together (protected)
│   ├── feat/auth-login (Member 2)
│   ├── feat/books-catalog (Member 3)
│   ├── feat/reservations-hold (Member 4)
│   └─ fix/agent-sse-timeout (Member 8) ... one short-lived branch per task
```

14.1 What every member needs installed

1. **Git** (git-scm.com). Check with `git --version`.
2. **Node.js 20 LTS** and `npm`. Check with `node -v`.
3. **A GitHub account** — send your username to Member 1 so you can be added.
4. **VS Code** (recommended) with the ESLint and Prettier extensions.
5. **GitHub CLI** (optional but handy): `cli.github.com`, then `gh auth login`.
6. **Ollama** — needed only by members 6, 7, 8 and anyone running the AI features locally.

Tell Git who you are (once per laptop):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com" # same email as your GitHub account
git config --global init.defaultBranch main
git config --global pull.rebase false # 'git pull' merges (simpler for beginners)
```

Log in to GitHub from the terminal: run `gh auth login` and follow the prompts, **or** create an SSH key (`ssh-keygen -t ed25519 -C "you@example.com"`) and paste the contents of `~/.ssh/id_ed25519.pub` into GitHub → Settings → SSH and GPG keys.

14.2 First-time setup on each member's laptop

1. Accept the GitHub invitation (check your email or `github.com/notifications`).
2. Clone the repo and install packages.
3. Copy the environment template and fill in the values shared privately.
4. Seed the database and start the app to confirm everything runs.
5. Do the **practice PR**: add your name to the team table in `README.md` (steps in 14.3). This is everyone's first PR, before any real code.

```
git clone https://github.com/<owner>/bookstore-clubs.git
cd bookstore-clubs
git checkout dev
npm install
cp .env.example .env # Windows PowerShell: copy .env.example .env
# open .env and paste the values shared privately
npm run seed -- --reset
npm run dev # server on :4000, client on :5173
```

14.3 Daily workflow — every task, every member

Step 1 — Start from the latest dev

Always begin from up-to-date code so your branch includes everyone's merged work.

```
git checkout dev
git pull origin dev
```

Step 2 — Create a branch for this one task

One task = one branch. Use the naming rules in 14.4.

```
git checkout -b feat/books-catalog
```

Step 3 — Write code and check what changed

Work only inside the folders you own. Check often.

```
git status      # which files changed
git diff        # exact lines changed
```

Step 4 — Commit small and often

A commit is a save point. Commit each working step with a clear message.

```
git add server/src/modules/books
git commit -m "feat(books): add GET /api/books with filters"
```

Step 5 — Bring in teammates' latest work

Do this at least once a day and always before opening a PR, so conflicts are found on your laptop, not on GitHub.

```
git fetch origin
git merge origin/dev      # fix conflicts if any (see 14.6)
```

Step 6 — Test locally

CI will run these anyway — catching failures here is faster.

```
npm run lint
npm test --workspace server
```

Step 7 — Push your branch

The first push needs -u; later pushes are just git push.

```
git push -u origin feat/books-catalog
```

Step 8 — Open a Pull Request into dev

On GitHub click **Compare & pull request**, base = dev, fill in the template, link the issue (Closes #12), and request a reviewer.

```
gh pr create --base dev --fill      # CLI alternative
```

Step 9 — Respond to review

Make the requested changes on the same branch, commit and push — the PR updates automatically. Reply to each comment.

```
git add .
git commit -m "fix(books): validate page param"
git push
```

Step 10 — Merge and clean up

When CI is green and it's approved, click **Squash and merge**. Then go back to dev for your next task.

```
git checkout dev
git pull origin dev
git branch -d feat/books-catalog
```

14.4 Naming rules

Thing	Format	Example
New feature branch	feat/<module>-<short-desc>	feat/reservations-atomic-hold
Bug fix branch	fix/<module>-<short-desc>	fix/chat-citation-filter
Tests / docs / setup	test/... · docs/... · chore/...	test/reservations-race · docs/git-guide
Commit message	type(module): what changed (present tense)	feat(agent): stream tool status over SSE
Commit types	feat, fix, test, docs, refactor, chore, style	refactor(books): move filters to service
PR title	Same as the main commit	feat(clubs): moderator creates meetings

Keep PRs small: ideally under ~300 changed lines and one feature. Small PRs get reviewed the same day; huge ones sit for days.

14.5 Reviewing a teammate's PR

1. Open the PR on GitHub → **Files changed** tab and read the diff.
2. Pull it to your laptop and run it if it changes behaviour.
3. Check the Definition of done list (Section 16): schema in shared/, permissions on the server, tests, no secrets.
4. Leave comments on specific lines. Be specific and kind: "This route needs requireVerified" rather than "wrong".
5. Choose **Approve** or **Request changes**. Never approve your own PR.

```
gh pr checkout 12                # pulls PR #12 into a local branch
# or without the CLI:
git fetch origin feat/books-catalog
git checkout feat/books-catalog
npm install && npm test --workspace server && npm run dev
```

14.6 Resolving a merge conflict

A conflict happens when your branch and dev changed the same lines. Git stops and marks the file. It is normal and fixable:

1. Run `git merge origin/dev`. Git lists the conflicted files.
2. Open each file in VS Code. You'll see markers like the example below, with buttons *Accept Current / Incoming / Both*.
3. Edit the section so it contains the correct final code, and delete all `<<<<<<<, =====, >>>>>>>` lines.
4. If unsure, ask the person who wrote the other side (`git log origin/dev -- <file>` shows who).
5. Mark it resolved, commit, run tests, push.

```
<<<<<<< HEAD                (your branch)
app.use('/api/books', booksRouter);
=====
app.use('/api/clubs', clubsRouter);
>>>>>>> origin/dev          (incoming from dev)

# correct result keeps BOTH lines:
app.use('/api/books', booksRouter);
app.use('/api/clubs', clubsRouter);

git add server/src/app.js
git commit                    # Git pre-fills a merge message
npm test --workspace server
git push
```

To abandon a messy merge and start again: `git merge --abort`.

package-lock.json conflicts

Don't edit it by hand. Take dev's version, reinstall, and commit the regenerated file:

```
git checkout --theirs package-lock.json
npm install
git add package-lock.json package.json
git commit
```

14.7 Files that need extra care

File / folder	Why	Rule
shared/schemas, shared/constants.js	Every module depends on them	Small separate PR, Member 1 reviews (CODEOWNERS), announce breaking changes
server/src/app.js	Everyone mounts a router here	Add one line for your router, nothing else
server/scripts/seed.js	Demo data for the whole team	Send your seed data file; Member 1 wires it in
package.json / package-lock.json	Dependencies for everyone	Install with <code>npm install <pkg> -w server (or -w client)</code> ; mention new packages in the PR
.env	Contains secrets	Never committed. Add new variable names to .env.example only

14.8 Common mistakes and how to fix them

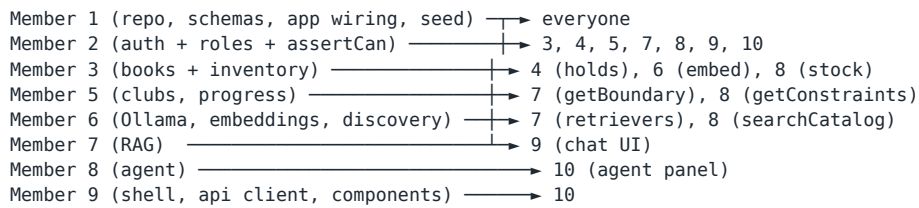
Problem	Fix
I made commits on dev instead of a branch	<code>git checkout -b feat/my-task</code> (takes the commits with you), then <code>git checkout dev && git reset --hard origin/dev</code>
I want to undo my last commit but keep the changes	<code>git reset --soft HEAD~1</code>
I want to throw away uncommitted changes to a file	<code>git restore <file></code>
I need to switch branch but have unfinished work	<code>git stash</code> → switch → come back → <code>git stash pop</code>
Push rejected: "updates were rejected"	<code>git pull origin <your-branch></code> , fix conflicts, push again. Never use <code>--force</code> on shared branches
I committed .env or a password	Tell Member 1 immediately. Remove the file (<code>git rm --cached .env</code>), commit, and change the password/secret — it stays in history
node_modules got committed	<code>git rm -r --cached node_modules</code> , check .gitignore, commit
CI failed on my PR	Open the failed check → read the red log line → fix locally → push again
My branch is very far behind dev	<code>git fetch origin && git merge origin/dev</code> , resolve, test, push

14.9 Never do this

- Push directly to main or dev (branch protection blocks it anyway).
- Use `git push --force` on any branch other than your own feature branch.
- Commit .env, passwords, API keys, the Atlas connection string or node_modules.
- Merge your own PR without a review, or merge with failing CI.
- Edit files in another member's folder without telling them — comment on their PR or message them instead.
- Leave a branch open for weeks. Merge small pieces often.

Quick reference: `git checkout dev` → `git pull` → `git checkout -b feat/x` → code → `git add` → `git commit` → `git merge origin/dev` → `npm test` → `git push` → open PR → review → squash & merge

15. Dependency map



Critical path: Member 1 wiring → Member 2 auth middleware → Members 3/5 services → Member 6 vectors → Members 7/8 AI → Members 9/10 screens. To keep it short, consumers code against mocks of the agreed function signatures from the start.

16. Definition of done and review checklist

- Route has Zod schema in shared/, validate() applied, correct middleware chain.
- Ownership/role checks on the server; private data filtered by owner in the query itself.
- Tests: happy path, permission failure, validation failure (plus race/boundary tests where relevant). CI green.
- Errors use AppError codes from shared/constants; no stack traces or secrets in responses.
- AI code: uses services not models, passes ctx.user, cites sources, handles AI_UNAVAILABLE.
- Frontend: loading/empty/error states, accessible labels, data-testid for E2E.
- Seed updated if your feature needs demo data; README/API notes updated.
- Reviewer checked the above and ran it locally.

17. Communication and tracking

What	How
Daily	5-line async update in the team group: done / doing / blocked
Twice a week	15-min stand-up call; blockers first
Task board	GitHub Projects: Backlog → In progress → Review → Done; one card per task with owner
Decisions	Short ADR file in docs/adr/ for anything that changes contracts or architecture
Help	Stuck > 2 hours → ask in group and tag the provider of the interface you depend on
Pairing	AI trio (6, 7, 8) and frontend pair (9, 10) pair once a week

Before you start: read Document 01 sections 8–11, clone the repo, run the seed, and reply in the team group with your name, the member number you're taking, and any questions about your section.